

Schema-Valid Does Not Mean Transformation-Correct

An Open Benchmark for ISO 20022 Information Preservation

Aksum Labs · Engineering technical report · 1 October 2026

Software: MessageBench 0.2.0-rc.1. This is a preprint, not a peer-reviewed publication. The corpus has automated expected-outcome checks but no authenticated independent human review. Corporate authorship includes coding-agent assistance. No production defect prevalence, institutional adoption or regulator endorsement is asserted.

Abstract

A financial-message transformation can produce a well-formed, schema-valid document while losing information that its implementer promised to preserve. Single-message validation and transformation correctness answer different questions. We present MessageBench, an offline benchmark and adapter-neutral oracle for declared information preservation. Versioned contracts specify field-level comparison semantics; exact-version extractors supply typed facts and source paths; reports expose examined, excluded, unsupported and unexamined information. The release candidate supports pacs.008.001.08 and pacs.002.001.10. Its original synthetic corpus contains 100 pairs with pre-recorded expected classifications. Automated evidence comprises 140 tests, 99 of 102 core semantic branches covered, 40 of 40 targeted mutants killed, agreement between two XSD processors across 200 documents, and bounded parser and contract fuzz campaigns. Six proof pairs include schema-valid identifier, reference, remittance and currency losses, while identity and permitted message-ID regeneration pass. These measurements establish a useful testing layer within the declared scope; they do not establish complete-document preservation, independent review, bank defect rates or operational safety.

1. Motivation

Integration engineers commonly need to answer whether an adapter preserved a reference, an account identifier or a repeated information item. The output may parse successfully and satisfy all structural constraints while still violating the adapter's intended relationship to its input. A schema describes valid documents, not an arbitrary adapter's preservation promise. Even a sophisticated single-message business-rule validator may have no basis for recovering a value omitted earlier in a pipeline.

This distinction matters where a system uses multiple message representations or software components. A reference shortened to another legal string remains structurally legal. An identifier converted through an integer may lose leading zeros without violating a general identifier type. A repeated remittance item may disappear while the remaining occurrence satisfies the schema. Such outcomes need a source/target relationship to be checked, unless the business context independently supplies the missing information.

Our motivation is engineering assurance, not an allegation about any institution. The cases are constructed to make the relevant distinction reproducible. They neither estimate how frequently these losses occur nor show that Ethiopian banks, national payment systems or specific vendors have them. We intentionally do not implement payment initiation, routing, settlement, screening, scheme rules or universal translation. The useful boundary is a bounded test that engineers can run on their own adapter outputs without granting another organization access to their systems.

2. Problem definition

Let S and T be source and target XML documents, V the exact supported message versions, and C a versioned preservation contract. For each assertion a in C , extraction supplies a typed fact $f_a(S)$ and $f_a(T)$, including explicit field presence and association information. A comparator r_a checks the declared relationship. A preservation result is therefore parameterized by the documents, versions, contract and extractor; it is not a property of the target in isolation.

The benchmark does not define a universal “correct transformation.” A message-ID regeneration may be legitimate under one contract and prohibited under another. Decimal values may compare numerically while a separately declared lexical requirement would compare their spelling. Repeated elements may be ordered or compared as a multiset. Contract authors must make these choices explicitly. A technically well-formed contract can still encode an inappropriate promise, so independent semantic review remains important.

Per-check states are PASS, FAIL, NOT_APPLICABLE, UNSUPPORTED and INDETERMINATE. A required unsupported or indeterminate check prevents overall PASS. The command exit codes distinguish assertion failure, configuration error, incomplete evidence, security/resource rejection and internal error. This separation makes it possible to tell a demonstrated violation from insufficient evidence. Every report warns: “Pass means only the listed assertions passed for these inputs and versions.”

3. Transformation correctness versus single-message validity

XML parsing verifies a syntactic prerequisite. XSD validation verifies structural and datatype constraints for a single document. Preservation checking evaluates a declared relationship between two documents. None replaces the others. We first validate the inputs against the exact namespace and reviewed local schema, then check facts covered by the contract.

The six proof pairs in evidence/baseline.json were run through XML parsing, lxml/libxml2 XSD validation, xmlschema validation and a selected Cognis lint baseline. Both sides of the semantic-loss pairs remain valid in those tested baselines. MessageBench reports the violated preservation assertions. Identity and permitted message-ID regeneration provide positive controls. This is a concrete comparison against inspected tools and versions, not a statement that all validators are incapable of relational testing.

Proof pair	Parser / exact XSD	Preservation result
Identity	Pass	PASS
Leading-zero account loss	Pass	FAIL: PRESERVE-DEBTOR-ACCOUNT
End-to-end reference truncated	Pass	FAIL: PRESERVE-END-TO-END-ID
Repeated remittance removed	Pass	FAIL: PRESERVE-REMITTANCE
Message ID regenerated under permissive contract	Pass	PASS
Currency altered	Pass	FAIL: PRESERVE-SETTLEMENT-AMOUNT

The XML/XSD tools do not have the contract's source value in their validation predicate. Their passing these documents is expected behavior, not a bug. A downstream business rule with additional external context could detect some losses; that would address a different or overlapping layer. We claim only a useful combination of declared pair semantics, reusable original fixtures, coverage accounting and reproducible redacted evidence.

4. Preservation contracts and comparison semantics

Contracts are JSON documents validated with JSON Schema Draft 2020-12. The model is closed and contains no arbitrary XPath, embedded code, downloaded regular-expression execution or plugin invocation. Public defaults are engineering contracts, not statements of national or scheme authority. Each assertion identifies a supported field, comparator, cardinality, missing-value policy and whether it is required. Source and target namespaces, contract version, provenance and exclusions make the test's scope explicit.

Identifiers remain strings. Account scheme and issuer context are separate facts; no invented Ethiopian check digit or assumed IBAN conversion is introduced. Decimal comparison uses Python Decimal, with currency compared as a separate semantic dimension. Instructed and settlement amounts are different fields. Text compares exact Unicode by default; trimming, case folding, transliteration and normalization are not implicit. Ethiopic strings are preservation data, not claims about a reviewed Amharic interface.

Repeated items retain multiplicity. Ordered-list semantics detect movement; multiset semantics permit reordering while retaining counts. Batch association requires unique declared keys and does not guess based on amount or a person's name. Ambiguous associations become INDETERMINATE. Dates and times preserve known schema semantics; unsupported calendar/timezone interpretation is not guessed. Message-ID regeneration is allowed only when the contract says so.

These policies matter for false positives as well as missed losses. Treating every changed identifier as loss could incorrectly reject an intentionally regenerated message identifier. Silently normalizing text could accept a transformation that changed meaningful bytes. The corpus therefore pairs negative cases with positive controls for allowed representation changes.

5. Architecture and coverage accounting

The runtime reads bounded regular files, checks resource limits, parses XML without external resolution, validates an exact root QName and local XSD, extracts version-specific facts, applies the contract and emits deterministic reports. There is no server, database, account system, telemetry, runtime network download or adapter execution. Engineers run the adapter separately and hand over the before/after files. This interface is independent of the adapter's implementation language.

Coverage is classified as examined, explicitly excluded, unsupported or unexamined. A check count is not a measure of full-document preservation. An extracted field may be excluded by a contract; a document may contain information outside the extractor's vocabulary. Reports preserve those distinctions rather than inferring that a successful subset covers the remainder. The initial release has 27 extractable fields and ten contracts containing 38 required assertion declarations; repeated declarations across contracts are not 38 globally unique information concepts.

Deterministic canonical JSON uses UTF-8, LF and stable ordering, with no timestamp in canonical result bytes. Hashes bind input and contract metadata; they are not anonymization. Default reports omit names, account numbers, payment values and raw XML. Text, escaped static HTML and JUnit provide additional views of the same scope-labelled result.

6. Corpus design

The paired corpus contains 100 original synthetic scenarios split across the initial pacs.008 corpus, extended pacs.008 semantics and pacs.002 status-message coverage. Expectations were recorded separately from implementation outcomes. Manifest entries describe the intended defect, comparator, positive control and provenance. Hash-bound packets allow reviewers to inspect the actual bytes and expected classifications. No authenticated independent reviewer decision is present.

Cases cover identity, QName-equivalent prefixes, leading-zero identifiers, account context, decimal and currency changes, reference truncation, repeated-field loss and duplication, Unicode alteration, allowed regeneration, optional absence, declared exclusions and batch association. A separate generated security/workflow suite contains 13 scenarios with controls, including foreign/unsupported namespaces, unsafe XML, resource attacks, malformed contracts and missing outputs. It must not be counted as 13 independently reviewed additional financial-message pairs.

Synthetic generation permits controlled interventions: one declared information property is changed while positive controls keep legitimate changes visible. The corpus is deliberately curated, not sampled from a bank population. Its classification accuracy demonstrates agreement with recorded expectations, not a measured rate of real operational defects. Vendor fixtures, customer messages, proprietary Ethiopian rules and paid usage guidelines are excluded.

7. Threat model

The principal trust boundaries are input files, catalog paths, XML parsing, contract configuration, extracted facts, report generation and CI dependencies. An attacker could supply malicious XML, exploit schema resolution, exhaust parser resources, inject HTML into reports or design a contract that produces misleading assurance. Inputs are bounded to 5 MiB, XML depth 64, 100,000 elements, 1 MiB per text node and 1,000 assertions under the initial profile.

DTD, external entities, XInclude and network resolution are disabled. Reviewed catalogs use local schemas and safe-root handling; the implementation does not execute arbitrary contract logic. Default output is redacted and static HTML is escaped. CI uses synthetic material, read-only contributor credentials and SHA-pinned actions. No adapter shell-command flag exists, avoiding a separate execution boundary entirely.

These controls are not an operating-system sandbox. Linux is the verified file-safety/process profile, and missing platform protections fail closed. Bounded fuzzing does not prove the absence of parser vulnerabilities, especially in native dependencies. Dependency inventories and native advisory review complement Python package scanning. Signed provenance authenticates the builder identity and artifact hashes; it does not validate corpus semantics or replace human review.

8. Experimental methodology

All reported numbers come from committed evidence. We distinguish the original local engineering measurements from subsequent hosted CI, and same-environment artifact equality from independent reproduction. Core branch coverage is measured for Aksum-authored comparison/extraction semantics; it is not a whole-program coverage percentage or an upstream dependency coverage claim.

The targeted mutation campaign modifies the required assertion behavior and checks the corresponding pre-recorded witness. It includes 40 tested mutants and verifies that every required assertion declaration has a failing witness. It is not a general-purpose mutation score over every possible source-code mutation. XML classification is checked against lxml/libxml2 and the separately implemented xmlschema processor; agreement is checked on both sides of the 100 pairs.

Parser fuzzing executes 100,000 bounded iterations and contract fuzzing 10,000 mutations. The campaigns use deterministic bounded inputs and explicit expected rejection behavior. We report completed campaigns and findings, not unbounded fuzz duration or security certification. Unit, property, golden, metamorphic, integration and security tests exercise contract and parser behavior. Test answers are not modified merely to match the implementation.

9. Results

Measurement	Observed result	Interpretation boundary
Paired synthetic corpus	100 cases, expected classifications matched	Curated expectations; no independent review
Pytest suite	140 passing tests	Tested implementation profile
Core semantic branch coverage	99/102, 97.06%	Selected authored semantic code
Targeted mutation witnesses	40/40 killed	Defined campaign, not universal mutation testing
Differential XSD	200 documents, all classifications agree	Two tested processors/versions
Parser fuzz	100,000 iterations completed	Bounded campaign; no proof of exhaustive safety
Contract mutations	10,000 completed	Bounded malformed/configuration inputs

The central proposition is directly demonstrated by schema-valid semantic-loss targets. Positive controls prevent the demonstration from becoming a detector that rejects all transformations. Incomplete evidence cannot yield a required PASS, and report scope prevents the benchmark from claiming an untested whole-document guarantee.

The evidence does not support precision/recall estimates over real defects. There is no independent ground-truth bank dataset, representative random sample or external adapter fleet. Reported expected classification is measured against curated synthetic cases. Generalization must be tested through future independent contributions and real local adapter use without uploading private financial messages.

10. Performance measurement

The committed performance run measures 1,000 small in-process comparisons while cycling the six proof pairs. It includes file reads, contract validation and schema compilation for each iteration, uses a warm operating-system cache and excludes CLI startup. The six distinct pairs total 12,844 input bytes. The host is an Intel Core i9-14900HX, Linux WSL2 with 16,375,448 kB reported memory, Python 3.12.3, lxml 6.1.3 and libxml2 2.15.4.

Observed throughput is 80.624 pairs per second, p50 latency 11.741 ms, p95 latency 15.584 ms and process peak RSS 40,360 KiB. RSS is the lifetime process peak, not incremental pair allocation. This is a single shared-host run with no confidence interval. It does not estimate large-batch throughput, production latency, scaling, an SLA or comparative performance against another project. A new hosted run must be labelled separately if performed.

The benchmark intentionally includes compilation overhead rather than presenting an optimized hot-oracle rate. Future performance work could separate schema cache effects, input size, batch association and report rendering. Such changes should preserve result determinism and resource controls, and should be measured rather than asserted as improvements.

11. Failure classes and controls

Identifier losses include dropped leading zeros, changed issuer/scheme context and shortened references. Amount losses include rounding and independent currency changes. Repeated-field losses include deletion and duplication under ordered or multiset contracts. Unicode changes test exact text rather than language interpretation. Positive controls include equal decimal values with changed lexical scale, allowed list reordering, optional absence on both sides and explicitly regenerated message IDs.

Association failures differ from information losses. A batch with nonunique declared keys does not justify a guessed pairing; the result is incomplete/indeterminate. Foreign namespaces with familiar local names also fail the exact-version requirement. Invalid targets are rejected as validation problems, not presented as the central schema-valid preservation example. Unsupported extensions and unexamined fields remain visible in coverage.

Security cases exercise DTD/entity rejection, remote resolution attempts, excessive depth/size, malformed contracts and HTML payload handling. The expected outcome is deterministic safe rejection or escaped reporting, with positive controls showing normal messages still work. Resource limits provide bounded processing, not a guarantee against every runtime or native-library defect.

12. Prior art

Provide ISO 20022 provides broad Java models, parsing and serialization; its inspected README distinguishes commercial validation capabilities. mx20022 provides Rust schema-based models, parsing, translation, round-trip tests and explicit loss warnings inside translations. iso20022-cbpr-ur offers individual-message usage-rule evaluation and rule/XPath coverage instrumentation. Mojaloop Testing Toolkit provides sophisticated API/scenario testing and onboarding. Cognis was used as a concrete selected lint baseline; only inspected/actually executed capabilities are attributed to it.

MessageBench does not invent XML validation, round-trip testing, contract testing, loss warnings or coverage reporting. Its bounded contribution is the combination of an adapter-independent pair oracle, explicit typed comparison policies, original reusable synthetic pairs, honest coverage classification and reproducible privacy-conscious evidence. Exact inspected commits and capabilities are in docs/prior-art.md and evidence/prior-art-snapshots.json. This is selected-source inspection, not a proof that no equivalent exists anywhere.

An upstream adapter can be used without taking a MessageBench dependency: emit source and target files and run the oracle externally. Conversely, independently useful synthetic preservation cases can be contributed to existing parsers/translators without imposing this architecture. The prepared upstream test package is not an accepted contribution or evidence of maintainer endorsement.

13. False assurance risks and limitations

A PASS can be misleading if a reader ignores the contract, unsupported facts or unexamined fields. A contract can encode the wrong promise. An extractor can omit information. A fixture corpus can overlook important interactions. A source/target comparison can be incorrectly associated. These risks motivate explicit coverage, required incomplete states, targeted mutations and genuine independent review, but do not disappear because the tests pass.

The release supports only two exact namespaces: urn:iso:std:iso:20022:tech:xsd:pacs.008.001.08 and urn:iso:std:iso:20022:tech:xsd:pacs.002.001.10. They are engineering choices, not assertions about versions used by EATS or EthSwitch. camt.053.001.08 remains excluded after unresolved joint-contributor redistribution rights. No national account format, calendar convention, proprietary rule or live connectivity is inferred.

Independent technical and rights review, outside use, sustained security response and neutral multi-maintainer governance are not established. The project remains 0.2.0-rc.1, not v0.5 or v1.0. Corporate bot attribution and an authentic signature cannot supply a fabricated reviewer. No foundation badge, acceptance, DOI or adoption may be inferred from the existence of prepared submission files.

14. Reproducibility

Clone <https://github.com/aksum-labs/messagebench> and record the exact commit. The hashed development lock and reviewed native build profile reproduce the documented test environment. `docs/reproduce.md` specifies each major evidence command. The wheelhouse supports fresh offline installation on the documented Linux x86_64 CPython 3.12 profile; that platform specificity must be disclosed.

A minimal demonstration runs messagebench on a source/target pair with the preservation contract. Corpus verification, differential XSD, mutation and security scripts reproduce their respective claims. Canonical results include hashes and versions with no wall-clock timestamp. Golden hashes guard against unintended output changes. The JavaScript file-handoff example is external-style, not independently adopted third-party software.

Two clean unsigned Python wheel/sdist builds were bit-identical in the original local environment. That is same-environment reproducibility, not an independent reproduction by another reviewer or host. Native wheel bit reproducibility is not claimed. Hosted build/signature evidence, when available, identifies its actual publication commit and workflow; it must not be confused with the historical local engineering snapshot.

15. Licensing and rights

Original MessageBench code, synthetic fixtures, contracts and documentation use Apache-2.0. Standard-schema assets retain separate SWIFTStandards-2005 terms and are included unmodified under the documented royalty-free supporting-software interpretation. Redistribution rights and OSI/FSF open-source classification are separate questions. The rights register lists source URLs, versions, hashes, terms and inclusion decisions. No paid ISO publication, proprietary SWIFT usage guideline, commercial BIC directory or private Ethiopian specification is redistributed.

camt.053 assets are excluded because the inspected public sources did not establish complete joint-contributor redistribution permission. This is a definitive release exclusion, not a claim that lawful support is globally impossible. A future release requires actual clearance before adding those bytes. Native dependency notices and source/relink instructions accompany the prepared Linux bundle; relevant LGPL obligations are not replaced by the project's Apache license.

16. Future work

The highest-priority next step is independent review of contract semantics, positive controls, expected answers and rights provenance. Useful contributions include new original preservation edge cases, reviewed association contracts and language-neutral examples with actual upstream adapters. External use should be demonstrated locally without publishing customer messages. Maintainer response history should be recorded as it occurs.

Expansion to new message versions depends on technical rigor and asset rights, not marketing breadth. General translation, payment operations and scheme certification remain outside scope. Future measurements should include separate hosts, input-size strata, association workloads and confidence intervals. Any foundation hosting proposal must preserve the code/asset rights boundary and require authorized human governance decisions.

17. Conclusion

Single-message schema validity does not establish a transformation's declared preservation relationship. The measured synthetic benchmark demonstrates that a bounded pair oracle can detect schema-valid information losses while accepting identity and explicitly allowed changes. MessageBench makes this layer inspectable through versioned contracts, exact extraction, coverage accounting and deterministic redacted evidence. Its usefulness is limited to those declared checks and tested versions. Independent review, broader use and durable maintenance remain necessary real-world work.

References and evidence

Repository: <https://github.com/aksum-labs/messagebench>. Reproduction: [docs/reproduce.md](#) and [paper/reproducibility-appendix.md](#). Baselines: [evidence/baseline.json](#). Corpus manifests: [corpus/index.json](#), [corpus/extended/index.json](#) and [corpus/pacs002/index.json](#). Branch evidence: [evidence/coverage.json](#) and [evidence/rule-coverage.json](#). Mutation evidence: [evidence/mutation-results.json](#). XSD comparison: [evidence/differential-xsd.json](#). Fuzz: [evidence/fuzz-extended.txt](#) and [evidence/contract-fuzz.json](#). Performance: [evidence/benchmark.json](#). Prior art: [docs/prior-art.md](#) with exact source commits. Rights: [evidence/rights-register.json](#) and [docs/standards-provenance.md](#).